

Feature Specification: EBS Project Management

Feature Branch: 001-ebs-project-management

Created: 2026-03-30

Status: Draft

Input: User description: "Build EBS-Project-Management that mimics Microsoft Project with modern sleek UI/UX and Excel-like flexibility for managing engineering project milestones, resources, and statistics across teams, members, projects (phases → milestones → epics → stories) with allocated resources per team."

User Scenarios & Testing (*mandatory*)

User Story 1 – Create and Structure a Project Hierarchy (Priority: P1)

As a project manager, I want to create a new project and have the system automatically generate placeholder phases, milestones, epics, and stories beneath it so that I can begin planning immediately without manually building every level of the hierarchy.

Why this priority: The project hierarchy (Project → Phases → Milestones → Epics → Stories) is the foundational data structure. Every other feature—resource allocation, timeline calculation, analytics—depends on this existing first.

Independent Test: Can be fully tested by creating a new project and verifying that placeholder sub-items appear at every level, then replacing placeholders with real items and confirming the hierarchy updates correctly.

Acceptance Scenarios:

1. **Given** I am on the Project view, **When** I create a new project with a name and description, **Then** the system creates the project and auto-generates one placeholder phase containing one placeholder milestone containing one placeholder epic containing one placeholder story.
2. **Given** a project with placeholder milestones exists, **When** I add a real milestone specifying its parent phase, **Then** the placeholder milestone is replaced (if one remains) and the phase's estimated points and dates adjust accordingly.
3. **Given** a project with no remaining placeholders at the milestone level, **When** I add another milestone to a phase, **Then** the phase expands its estimated points and recalculates its end date.
4. **Given** I insert a new phase or milestone at an arbitrary position in the sequence, **When** the item is confirmed, **Then** all subsequent items are reordered and affected start/end dates are recalculated.
5. **Given** a milestone exists with an end date and the next milestone has a different start date leaving a gap, **When** the system detects this gap, **Then** it automatically inserts a placeholder milestone to fill the gap, and the same rule applies for phases.

User Story 2 – Allocate Resources to Milestones and Track Allocation Health (Priority: P1)

As a project manager, I want to assign team members to specific milestones with a contribution percentage and velocity estimate, and immediately see warnings when a resource is over-allocated (total

allocation > 100%) or under-utilized (total allocation < 100%).

Why this priority: Resource allocation directly drives timeline calculations and is the core differentiator for project planning. Without it, end-date estimates and velocity analytics are impossible.

Independent Test: Can be fully tested by creating a team with members, assigning members to milestones with varying percentages, and verifying that allocation warnings appear at the correct thresholds and that velocity figures compute correctly.

Acceptance Scenarios:

1. **Given** a member has a base allocation of 100%, **When** I assign them to milestones whose contribution percentages sum to more than 100%, **Then** a red warning badge appears on that member indicating over-allocation, displaying the total allocation percentage.
 2. **Given** a member's total allocation across **active** milestones is less than 100%, **When** I view the member on the Resource page, **Then** a yellow warning badge appears indicating under-utilization.
 3. **Given** a member is assigned to multiple active milestones, **When** I view their profile, **Then** I see their overall average velocity (mean of their estimated velocity across all active milestones) and a contribution matrix listing each active milestone, their velocity for it, and their percentage of contribution.
 4. **Given** a milestone has allocated resources, **When** I view the milestone detail, **Then** I see a participation matrix listing each resource, their velocity if 100% dedicated, their actual contribution percentage, and their average FTO (Finish-Time Offset).
 5. **Given** a milestone is in a non-active state (Not Started, On Hold, Completed, or Cancelled), **When** the system calculates a member's velocity and allocation, **Then** that milestone is excluded from the calculation.
-

User Story 3 – View and Edit the Project Management Dashboard (Priority: P1)

As a project manager, I want a central Project Management view where I can select a project milestone and see its contribution matrix, timeline, and analytics, and where I can perform all editing operations (add projects, phases, milestones, allocate resources, change allocations) with full flexibility similar to an Excel spreadsheet.

Why this priority: This is the primary workspace where daily management happens. It brings together hierarchy, resources, and timelines in one place with inline editing, making the tool usable for real planning work.

Independent Test: Can be fully tested by navigating to the Project Management view, selecting a milestone, verifying the contribution matrix and analytics display, then performing inline edits (adding a phase, reassigning a resource) and confirming recalculations propagate.

Acceptance Scenarios:

1. **Given** I am on the Project Management page, **When** I select a project milestone, **Then** I see three sections: (a) Contribution Matrix table showing member, velocity, and percentage for that milestone; (b) Milestone Timeline visualization; (c) Analytics panel showing total number of resources, average velocity, and estimated end date.
-

2. **Given** I am on the Project Management page, **When** I select multiple project milestones, **Then** I see a combined view with separate panels or a consolidated comparison for each selected milestone.
 3. **Given** I am viewing the whole-project timeline, **When** I look at any component (phase, milestone, epic), **Then** I see the resource allocation breakdown for that component.
 4. **Given** I am on the Project Management page, **When** I add a new project, phase, milestone, or allocate/change a resource, **Then** the system recalculates end dates, allocation percentages, and velocities for all affected components in real time, while preserving original start dates and original estimated end dates.
 5. **Given** a recalculation changes the end date of any item, **When** the new end date is saved, **Then** the system records the reason for the change (e.g., "Resource X allocation changed from 50% to 30%" or "New epic added to milestone Y").
 6. **Given** I am editing inline, **When** I add a row for a new resource or component, **Then** a placeholder row appears that I can fill in, similar to adding a row in a spreadsheet.
-

User Story 4 – Resource Page: View and Edit Team Members (Priority: P2)

As a project manager, I want a dedicated Resource page where I can view all team members, their allocation status, overall velocity, and contribution matrix, and where I can edit member details and assignments.

Why this priority: While resource data feeds into the Project Management dashboard, a dedicated Resource view gives focused management of people and teams without the distraction of project hierarchy. It depends on the allocation model from P1 stories.

Independent Test: Can be fully tested by navigating to the Resource page, viewing the list of members with their allocation badges, editing a member's assignment, and confirming the changes reflect in their velocity and contribution matrix.

Acceptance Scenarios:

1. **Given** I am on the Resource page, **When** it loads, **Then** I see a list of all team members with their name, team, allocation percentage (with red/yellow warning indicators), and overall average velocity.
 2. **Given** I select a member, **When** their detail panel opens, **Then** I see their full contribution matrix: each active milestone they participate in, their estimated velocity for that milestone, and their contribution percentage.
 3. **Given** I edit a member's contribution percentage for a specific milestone, **When** I save the change, **Then** the system recalculates the member's overall velocity, their allocation percentage, and the affected milestone's participation matrix and end date.
 4. **Given** I want to add a new member, **When** I click add, **Then** a placeholder row appears where I can enter the member's details and optionally assign them to milestones immediately.
-

User Story 5 – Project Hierarchy View and Inline Editing (Priority: P2)

As a project manager, I want a dedicated Project view showing the full hierarchy (Project → Phases → Milestones → Epics → Stories) in a tree/grid layout where I can expand, collapse, reorder, and edit any item

inline.

Why this priority: A focused project hierarchy view gives detailed structural management. It complements the broader Project Management dashboard by providing a deep-dive editing experience for the project structure.

Independent Test: Can be fully tested by viewing a project's full hierarchy, expanding/collapsing levels, reordering milestones via drag-and-drop or context menu, and verifying recalculations occur.

Acceptance Scenarios:

1. **Given** I am on the Project view, **When** I select a project, **Then** I see its full hierarchy in a tree/grid with columns for description, total estimated points, number of allocated resources, state, original start date, original end date, actual start date, and adaptive end date.
 2. **Given** I am viewing a project hierarchy, **When** I drag a milestone to a different position within its phase, **Then** the milestone is reordered and all affected items' start and end dates recalculate.
 3. **Given** I want to reorder without drag-and-drop, **When** I right-click a milestone and choose "Move Up" or "Move Down," **Then** the same reordering and recalculation occurs.
 4. **Given** I edit the estimated points of an epic, **When** I save, **Then** the parent milestone's total estimated points update and the adaptive end date recalculates.
 5. **Given** an item has a state field, **When** I change a milestone's state to "on-hold" or "completed," **Then** the system excludes that milestone from active velocity and allocation calculations for all assigned resources.
-

User Story 6 – Team Management (Priority: P2)

As a project manager, I want to create and manage teams, assign members to teams, and see which projects each team is responsible for.

Why this priority: Teams are the organizational container for members. Managing team composition is necessary for the Resource page and for understanding team-level project responsibility.

Independent Test: Can be fully tested by creating a team, adding members, assigning team responsibility for projects, and verifying the relationships display correctly.

Acceptance Scenarios:

1. **Given** I am on the Teams section, **When** I create a new team with a name, **Then** the team appears in the list and I can add members to it.
 2. **Given** a team exists, **When** I assign a project to that team's responsibility, **Then** the project appears under the team's project list.
 3. **Given** a team has members, **When** I view the team detail, **Then** I see all members with their allocation status and the set of projects the team is responsible for.
-

User Story 7 – Timeline Calculation Engine with Change Tracking (Priority: P1)

As a project manager, I want the system to automatically calculate end dates based on estimated points, resource velocity, and allocation percentages, and to preserve original estimates while showing adaptive

dates when changes occur, along with a log of what caused each date change.

Why this priority: Automated timeline calculation is the core computational engine that makes this tool valuable beyond a static spreadsheet. Without it, users must manually compute dates, defeating the purpose of the tool.

Independent Test: Can be fully tested by setting up a milestone with resources and points, verifying the calculated end date, then changing an allocation and confirming the adaptive end date updates with a change reason logged.

Acceptance Scenarios:

1. **Given** a milestone has estimated points, allocated resources with velocities and contribution percentages, and a start date, **When** the system calculates, **Then** it produces an estimated end date with supporting statistics (e.g., effective velocity, expected duration).
2. **Given** a milestone has an original calculated end date, **When** I change a resource's allocation or add/remove an epic, **Then** the adaptive end date recalculates but the original start date and original estimated end date remain unchanged.
3. **Given** an adaptive end date changes, **When** the change is saved, **Then** the system records a change-log entry with the reason (e.g., "Allocation of Member A changed from 80% to 50%", "Epic 'Payment Integration' added with 40 story points").
4. **Given** a child item's dates change, **When** the parent's dates are affected, **Then** the parent's adaptive end date also recalculates and the cascade continues upward through the hierarchy.
5. **Given** a start date for an item is not explicitly set, **When** the system resolves dates, **Then** the start date is inherited from the parent item.

Edge Cases

- What happens when a user removes the last resource from an active milestone? The system should warn that no resources remain and set velocity to zero, making the end date indeterminate until resources are reassigned.
- What happens when a member is assigned to zero active milestones? Their allocation shows 0% with a yellow under-utilization warning and their average velocity displays as N/A.
- What happens when all milestones under a phase are set to non-active states? The phase should reflect that it has no active work and resource calculations for that phase return zero.
- What happens when a user reorders items such that a circular dependency could theoretically arise in date inheritance? Since the hierarchy is strictly parent-child (Project → Phase → Milestone → Epic → Story), circular dependencies are structurally impossible; the system enforces this hierarchy.
- What happens when a user deletes a non-placeholder item that has child items? The system should warn that all child items will also be removed and require confirmation before proceeding.
- What happens when two managers simultaneously edit the same milestone's resource allocation? The last save wins, but the system displays a notification that the data has been updated by another user and offers to refresh.
- What happens when estimated points for a story are set to zero? The story is valid but contributes nothing to the parent epic's point total; no special warning is needed.
- What happens when a gap-filling placeholder milestone is manually edited? It becomes a real milestone and is no longer treated as an auto-generated placeholder.

- What happens when the user tries to add a child below the story level? The system should not allow it; stories are the leaf level of the hierarchy.
- What happens when a milestone with allocated resources is deleted? All resource allocations are cascade-released, affected members' allocation percentages and velocities recalculate, and the change is logged with the deletion reason.

Requirements (*mandatory*)

Functional Requirements

Hierarchy Management

- **FR-001:** System MUST enforce a strict hierarchy: Project → Phase → Milestone → Epic → Story. No other nesting is permitted.
- **FR-002:** When a user creates an upper-level item (e.g., a project), the system MUST auto-generate one placeholder item at every level below it down to the story level.
- **FR-003:** When a user creates a lower-level item and specifies its parent, the system MUST replace an existing placeholder at that level (if one remains) or expand the parent's estimated points and end date if no placeholder remains.
- **FR-004:** Users MUST be able to insert a phase or milestone at any arbitrary position within its parent, and the system MUST reorder siblings and recalculate affected dates and statistics.
- **FR-005:** When a gap exists between consecutive milestones (or phases) such that one ends before the next begins, the system MUST automatically insert placeholder items to fill the gap.
- **FR-005a:** Milestones within a phase are sequential by default — each milestone's start date equals the previous milestone's end date. A milestone MAY be explicitly marked as parallel, allowing it to overlap with adjacent milestones. Gap-filling (FR-005) applies only to sequential milestones.
- **FR-006:** Stories are the leaf level; the system MUST NOT allow creating child items beneath a story.

Resource & Allocation Management

- **FR-007:** Each member MUST belong to exactly one home team. The system MUST calculate actual allocation as the sum of contribution percentages across all active milestones, regardless of which team owns the milestone's project.
- **FR-008:** When a member's total allocation exceeds 100%, the system MUST display a red over-allocation warning on that member.
- **FR-009:** When a member's total allocation is below 100%, the system MUST display a yellow under-utilization warning on that member.
- **FR-010:** The system MUST calculate a member's overall average velocity as the mean of their estimated velocities across all active milestones they participate in.
- **FR-011:** Each member MUST have a contribution matrix showing, for each active milestone: the milestone name, estimated velocity (story points per week), and contribution percentage.
- **FR-012:** Each hierarchy item MUST use one of five states: Not Started, Active, On Hold, Completed, Cancelled. Only items in the "Active" state contribute to resource velocity and allocation calculations. Items in Not Started, On Hold, Completed, or Cancelled states MUST be excluded from these calculations.
- **FR-013:** Each hierarchy item (project, phase, milestone, epic, story) MUST expose a participation matrix listing each allocated resource, their velocity if 100% dedicated, their actual contribution

percentage, and their average FTO.

Timeline & Calculation Engine

- **FR-014:** Each hierarchy item MUST have: description, total estimated points, number of allocated resources, state, original start date, original calculated end date (with calculation statistics), actual start date, and adaptive end date.
- **FR-015:** The system MUST calculate end dates using the additive velocity model with an efficiency buffer: $\text{milestone effective velocity} = \text{sum of each resource's (velocity} \times \text{contribution percentage)} \times \text{efficiency factor}$. End date = start date + (total estimated points ÷ buffered effective velocity). The efficiency factor (default 1.0, configurable per milestone, range 0.1–1.0) accounts for the fact that multiple resources do not achieve 100% parallel throughput. A lower efficiency factor produces a more conservative (later) end date.
- **FR-015a:** Each milestone MUST expose an editable efficiency factor (buffer). Changing the efficiency factor MUST trigger end-date recalculation and log the change reason.
- **FR-016:** When any input changes (allocation, velocity, scope), the system MUST recalculate adaptive end dates for all affected items while preserving original start dates and original estimated end dates.
- **FR-017:** Every change to an adaptive end date MUST be logged with a human-readable reason describing what caused the change (e.g., resource allocation modification, scope addition).
- **FR-018:** Start dates for child items MUST be inherited from the parent item unless explicitly overridden.

Views & User Interface

- **FR-019:** The system MUST provide a Resource page where users can view and edit team members, see allocation warnings, overall velocity, and contribution matrices.
- **FR-020:** The system MUST provide a Project view where users can view and edit the full project hierarchy in a tree/grid layout with expand/collapse, inline editing, and reordering.
- **FR-021:** The system MUST provide a Project Management dashboard where users can:
 - (a) Select a project milestone and see its contribution matrix, timeline, and analytics (total resources, average velocity, estimated end date).
 - (b) View multiple project milestones simultaneously.
 - (c) View the whole project timeline with resource allocation per component.
 - (d) Perform inline edits (add projects, phases, milestones, allocate resources, change allocations) that trigger immediate recalculation.
- **FR-022:** Core actions (adding items, reordering, editing, allocating resources) MUST be accessible through multiple interaction paths: inline editing, context menus, explicit buttons, and drag-and-drop where applicable.
- **FR-023:** The system MUST support Excel-like flexibility: placeholder rows for easy adding, inline cell editing, and the ability to introduce new entities or resources without navigating away from the current view.
- **FR-024:** The system MUST provide clear visual feedback for all interaction states: success, loading, empty, error, hover, active, invalid-target (for drag-and-drop), and completion.

Team Management

- **FR-025:** Each team **MUST** have a name, a list of members, and a set of projects the team is responsible for.
- **FR-026:** The system **MUST** provide a way to create, edit, and delete teams and manage team membership.

Change Tracking

- **FR-027:** The system **MUST** maintain a change log for every item that tracks date modifications with the reason for each change.

Deletion Behavior

- **FR-028:** When a hierarchy item is deleted, the system **MUST** cascade-release all resource allocations associated with that item and its children, automatically recalculate affected members' total allocation percentages and velocities, and log a change entry with the reason (e.g., "Milestone 'Alpha Release' deleted — 3 resource allocations released").
- **FR-029:** Before deleting a non-placeholder item with children or resource allocations, the system **MUST** display a confirmation warning listing the number of child items and resource allocations that will be affected.

Key Entities

- **Team:** An organizational group that owns a set of projects. Contains members and has a name.
- **Member:** A person belonging to exactly one home team. Key attributes: name, home team, base allocation (default 100%), computed total allocation percentage, overall average velocity, and contribution matrix (one entry per active milestone with milestone reference, velocity, and contribution percentage). A member can be allocated to milestones of any project regardless of which team owns the project.
- **Project:** The top-level planning container. Contains phases. Has description, total estimated points (aggregated), state, dates (original start, original end, actual start, adaptive end), and a participation matrix.
- **Phase:** A grouping within a project. Contains milestones. Has the same attribute set as Project. Phases are sequential within a project.
- **Milestone:** The primary planning unit within a phase. Contains epics. Has the same attribute set plus calculation statistics for original end date. Milestones have a state that determines whether they are "active" for calculation purposes.
- **Epic:** A grouping of stories within a milestone. Contains stories. Has the same attribute set as Milestone.
- **Story:** The leaf-level work item within an epic. Has description, estimated points, state, dates, and participation matrix. Cannot have children.
- **Change Log Entry:** A record associated with any hierarchy item that captures: timestamp, previous end date, new end date, and human-readable reason for the change.
- **Placeholder:** An auto-generated item at any hierarchy level that serves as a slot for future real items. Behaves like a regular item but is visually distinguished and can be replaced when a real item is created.

Success Criteria (*mandatory*)

Measurable Outcomes

- **SC-001:** Users can create a full project hierarchy (project with phases, milestones, epics, and stories) within 5 minutes on first attempt using auto-generated placeholders.
- **SC-002:** Resource allocation warnings (red for over-allocation, yellow for under-utilization) appear within 2 seconds of any allocation change.
- **SC-003:** When a resource allocation or scope change is made, all affected end dates, velocities, and statistics recalculate and display within 3 seconds.
- **SC-004:** Users can perform any core action (add item, reorder, edit, allocate resource) through at least two distinct interaction paths (e.g., inline edit and context menu).
- **SC-005:** Every adaptive end-date change includes a recorded reason that is visible in the item's change log.
- **SC-006:** Users can manage 50+ team members across 10+ projects with 100+ milestones without perceivable lag in navigation or editing.
- **SC-007:** The Project Management dashboard displays contribution matrix, timeline, and analytics for a selected milestone in a single screen without scrolling for standard data volumes (up to 15 resources per milestone).
- **SC-008:** Users can view and compare multiple milestones side by side on the Project Management page.
- **SC-009:** Gap-filling placeholder milestones are auto-inserted within 2 seconds of detecting a timeline gap between consecutive milestones.
- **SC-010:** 90% of first-time users can complete the primary workflow (create project → add milestone → allocate resource → view calculated end date) without external guidance.

Clarifications

Session 2026-03-30

- Q: Do milestones within a phase execute sequentially or in parallel? → A: Sequential by default with optional parallel override — milestones are sequential unless explicitly marked as parallel.
- Q: How do multiple resources combine for milestone end-date calculation? → A: Sum of effective velocities — milestone effective velocity = sum of all (individual velocity × contribution%). End date = start date + (total points ÷ summed effective velocity).
- Q: Can a member belong to multiple teams or only one? → A: Single home team with cross-team allocation — a member belongs to exactly one team but can be allocated to milestones of any project regardless of team ownership.
- Q: What is the definitive set of states for hierarchy items and which are calculative? → A: 5 states: Not Started, Active, On Hold, Completed, Cancelled. Only "Active" counts toward resource velocity and allocation calculations.
- Q: What happens to resource allocations when a hierarchy item is deleted? → A: Cascade-release — deletion removes all resource allocations for that item, automatically recalculates affected members' total allocation and velocity, and logs the change with reason.

Assumptions

- Users are primarily desktop web users (engineering managers / project managers) working on screens 1080p or larger.

- The application is single-tenant per deployment; multi-tenancy is out of scope for the initial release.
- Member velocity values are provided by the user as estimates; the system does not auto-derive velocity from historical data in this version.
- "Active" is the only state that counts toward velocity and allocation calculations. The complete state set is: Not Started, Active, On Hold, Completed, Cancelled. All non-Active states are excluded from calculations.
- FTO (Finish-Time Offset) is a user-provided estimate per resource per milestone; the system stores and displays it but does not auto-calculate it in this version.
- Concurrent editing by multiple users follows a last-write-wins model with refresh notifications; real-time collaborative editing (e.g., operational transforms) is out of scope.
- Authentication and authorization are handled by a standard session-based mechanism; detailed role-based access control is deferred to a later phase.
- The initial release focuses on a single calculation model for end dates ($\text{total points} \div \text{buffered effective velocity}$, where $\text{buffered effective velocity} = \text{summed velocities} \times \text{efficiency factor}$). The efficiency factor defaults to 1.0 (optimistic, full parallelism) and can be lowered per milestone to account for coordination overhead. Alternative scheduling algorithms (e.g., critical path) may be added later.
- Drag-and-drop is supported as a convenience for reordering; all drag-and-drop actions are also available via context menu or explicit controls as fallback.
- Data export (e.g., to Excel or PDF) is not in scope for this initial feature but the data model should not preclude it.